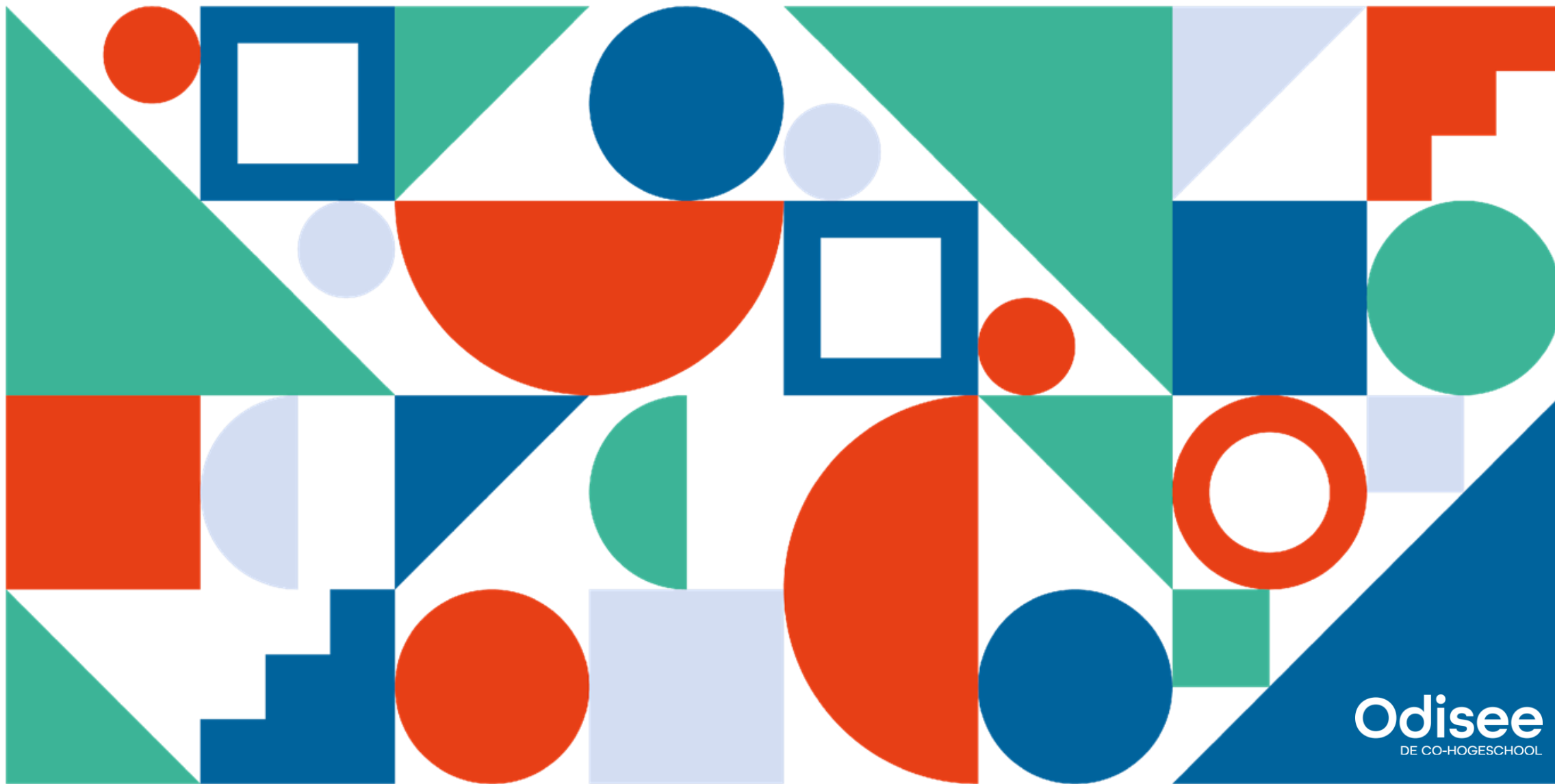




Odissee
DE CO-HOGESCHOOL

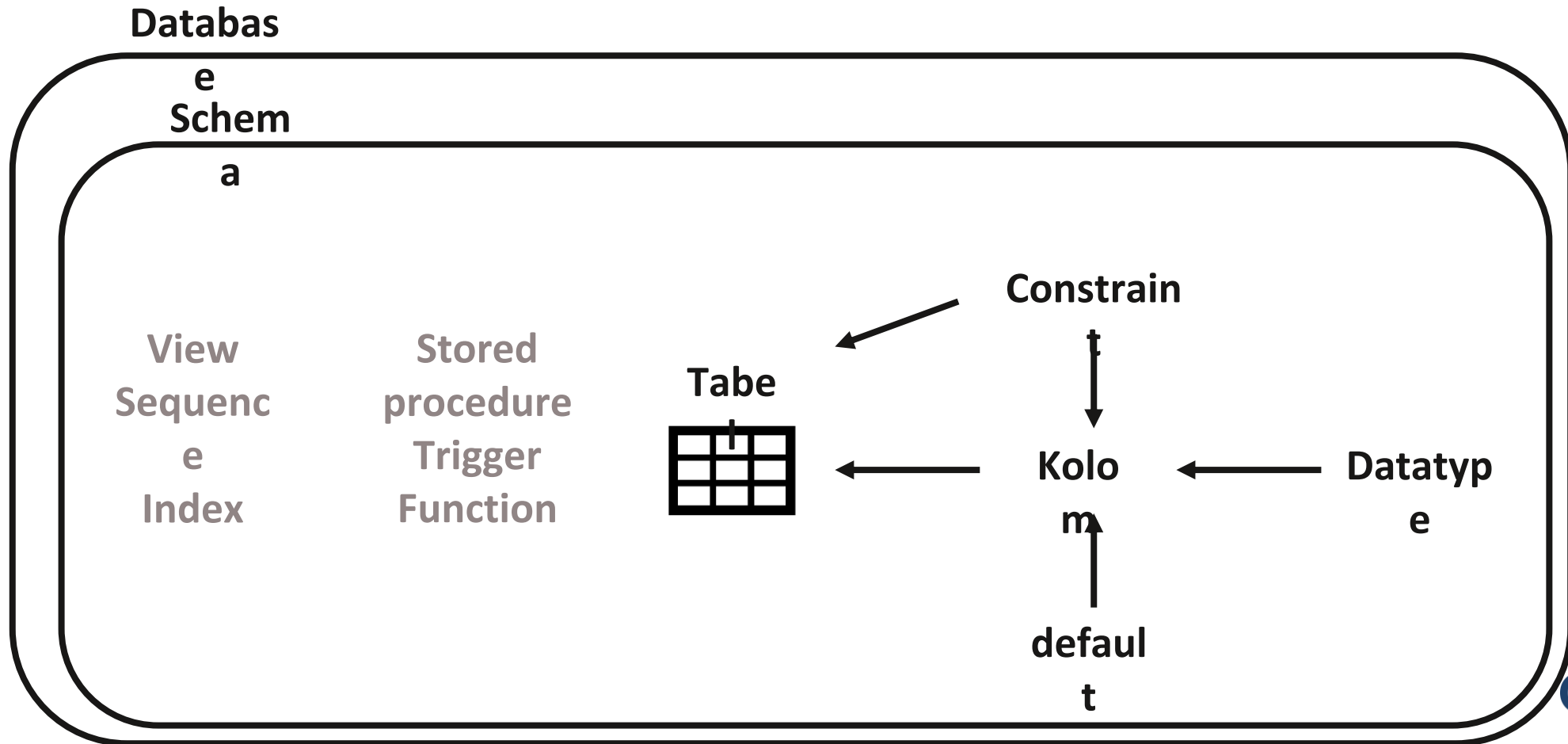


1.

DDL script

- **DML != DDL**
- **Data Manipulation Language**
 - **SELECT, INSERT, UPDATE, DELETE** data
 - Is van toepassing op de data
- **Data Definition Language**
 - **CREATE, ALTER, DELETE, TRUNCATE** table/database
 - Is van toepassing op de structuur = DB objects

Uit welke elementen bestaat de DB structuur?



- Script = opeenvolging van statements/queries in een specifieke volgorde die achter elkaar uitgevoerd worden.
- Handig om een hele database met tabellen en data in 1 actie aan te maken.
- Vb. de .sql files die je aan het begin van de cursus importeerde

```
INSERT INTO woningen VALUES (1,'Stationsstraat', 'B1080',3 , 'Villa',250000);
INSERT INTO woningen VALUES (2,'Gemeenteplein', 'B1080',2 , 'Appartement',210000);
INSERT INTO woningen VALUES (3,'Ringweg', 'NL0090',3 , 'Villa',350000);

-- c) Koper Mezelf brengt vandaag een bod uit van 220000 op huis 1 en 2. Koper Jij
INSERT INTO biedingen VALUES (1,1,220000,CURDATE());
INSERT INTO biedingen VALUES (1,2,220000,CURDATE());
INSERT INTO biedingen VALUES (2,2,250000,DATE_SUB(CURDATE(),INTERVAL 1 DAY));
INSERT INTO biedingen VALUES (2,3,250000,DATE_SUB(CURDATE(),INTERVAL 1 DAY));
```

2. Database/Schema

CREATE DATABASE [IF NOT EXISTS] *databasenaam*;

- Om een nieuwe database te maken
- Moet een unieke naam hebben in je DBMS

Schema $\sim$ Database

IF (NOT) EXISTS

- Een controle die valideert of een databank bestaat (of niet) alvorens we iets doen.
- Hierdoor voorkomen we dat we een fout krijgen wanneer de databank niet of wel al bestaat.
- `CREATE DATABASE mijndb;`
Geeft een error als mijndb al bestaat.
- `CREATE DATABASE IF NOT EXISTS mijndb;`
Geeft geen error als mijndb al bestaat.
mijndb DB zal gemaakt worden als ze nog niet bestaat.
- Laat toe een script meerdere keren uit te voeren, vb om de databank te “resetten”.

DROP DATABASE [IF EXISTS] *databasename;*

- Het **DROP DATABASE** statement wordt gebruikt om een database te verwijderen
- LET OP: Denk na voor je een database verwijderd. Het verwijderen van een database resulteert in het volledig verliezen van de data in de database. Dit is onomkeerbaar. (Tenzij je zo slim bent een backup te maken)

USE *database***name;**

- Met het use keyword kunnen we de database instellen waarop alle volgende statemens op moeten uitgevoerd worden.
- 1 Database per connectie
- Andere DB aan te spreken met qualifier
SELECT worlddb.countries.name

Oefening More Immo

- Start met het moreImmo script in Toledo en wijzig de code voor een nieuwe database 'MoreImmo'.
- Schrijf statements voor het aanmaken en droppen van de database.
- Maak ook een use statement.
- Je kan je inspireren op het employeesDB script.

2. Table

BASIC Commands

- **CREATE**
- **DROP/TRUNCATE**
- **ALTER**

CREATE Syntax

```
CREATE TABLE tabelnaam (  
    kolomnaam1 datatype,  
    kolomnaam2 datatype,  
    kolomnaam3 datatype,  
    ...  
);
```

CREATE Syntax

```
CREATE TABLE tabelnaam (  
    kolomnaam1 datatype [NOT NULL] [DEFAULT value1],  
    ...  
    [[CONSTRAINT tabelnaam_PK] PRIMARY KEY (kolomnaam,...)],  
  
    [[CONSTRAINT tabelnaam_relatie_FK] FOREIGN KEY  
        (kolomnaam,...) REFERENCES tabelnaam(kolomnaam,...)],  
    ...  
);
```

DROP/TRUNCATE

- Drop => verwijderd table
- Truncate => verwijderd table data
- Let op: Verwijderde gegevens kunnen we niet terughalen!

DROP Syntax

```
DROP TABLE tabelnaam;
```

TRUNCATE syntax

```
TRUNCATE TABLE tabelnaam;
```

Oefening MoreImmo

- **Wijzig de bestaande SQL code van woningen.**
- **Addres: We splitsen de gemeentenaam af in een nieuwe kolom. Bedenk zelf een correct datatype.**
- **Kamers zijn niet meer verplicht.**
- **De omschrijving veranderen we in een kolom categorie met als datatype VARCHAR(30).**
- **De vraagprijs heeft 9 cijfers voor de komma.**

3. Constraints

- Letterlijk: beperkingen
- Zijn beperkingen van allerlei aard welke een beperking/vorm/regel opleggen aan de data in de tabellen.
- Deze worden bepaald wanneer de tabellen gecreëerd worden met het **CREATE TABLE** statement, of nadat de tabel gecreëerd is met het **ALTER TABLE** statement.
- Welke beperkingen op de data in de tabellen ken je?

Wel Constraints

- NOT NULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY
- CHECK

Toch geen constraints

- **DEFAULT**
- **INDEX**
- **AUTO INCREMENT**
- **Data type**

Column level constraint Syntax

```
CREATE TABLE tabelnaam(  
    kolomnaam1 datatype constraint,  
    kolomnaam2 datatype constraint,  
    kolomnaam3 datatype constraint,  
    ....  
);
```


Column level constraint Syntax voorbeelden

```
CREATE TABLE personen(  
    RR NUMERIC(11) PRIMARY KEY,  
    email VARCHAR(60) UNIQUE,  
    geboortedatum DATE NOT NULL,  
    ....  
);
```

Table level constraint Syntax

```
CREATE TABLE tabelnaam(  
    kolomnaam1 datatype constraint,  
    kolomnaam2 datatype constraint,  
    kolomnaam3 datatype constraint,  
    CONSTRAINT_TYPE(columns),  
  
    CONSTRAINT tabel_constraintnaam_type  
        CONSTRAINT_TYPE(columns),  
  
    ...  
);
```

Constraint naming

Volgens de coding guidelines krijgen constraints een naam bestaande uit 3 stukken:

tabel_constraintnaam_type

vb employees_positiefSalaris_CC

De types zijn PK (primary key), FK (foreign key), NN (not null), UN (unique) en CC (check constraint).

Een PK is een uitzondering omdat de naam vaak wordt weggelaten, vb employees_PK.

Table level constraint Syntax voorbeelden

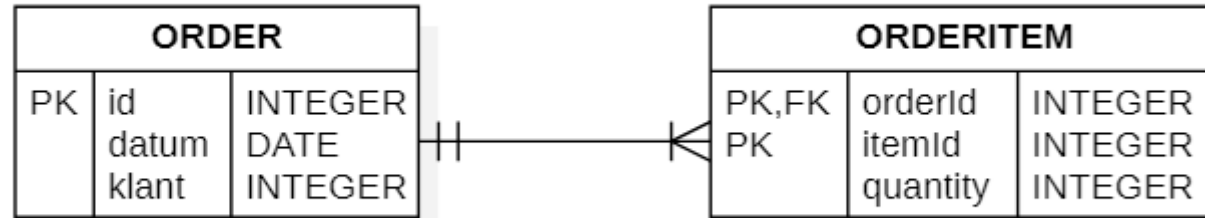
```
CREATE TABLE inschrijvingen(  
  RR_sporter NUMERIC(11),  
  sporttak VARCHAR(60),  
  inschrijvingsjaar NUMERIC(4),  
  
  PRIMARY KEY(RR_sporter, sporttak, inschrijvingsjaar),  
  
  CONSTRAINT inschrijvingen_verledenjaar_CC  
    CHECK(inschrijvingsjaar >= EXTRACT(YEAR FROM CURRENT_DATE()))  
  
  ....  
);
```



- De **PRIMARY KEY** constraint zorgt er voor dat we 1 record uniek kunnen identificeren.
- Primary keys moeten dus **UNIQUE** waardes hebben, en kunnen **niet NULL** zijn. Dit is impliciet!
- Een tabel kan slechts **ÉÉN primary key hebben**. In de tabel kan deze primary key uit 1 of meerdere kolommen(velden) bestaan table.
- In het ERD schema zijn alle velden met een sleutel, deel van de primary key

```
CREATE TABLE Eigenaar (  
    id INTEGER PRIMARY KEY,  
    naam VARCHAR(255) NOT NULL,  
    email VARCHAR(255) UNIQUE,  
    geboortedatum DATE  
);
```

Primary key met meerdere velden



```
CREATE TABLE orderItem (
    orderId INTEGER,
    itemId INTEGER,
    quantity INTEGER,
    PRIMARY KEY (orderId, itemId)
```

)

- De **CHECK** constraint wordt gebruikt om het **aantal waarden te limiteren** die in een kolom kunnen geplaatst worden a.h.v. een **conditie**.
- Wanneer je een **CHECK** constraint voor een **enkele kolom** instelt, worden enkel de waarden die voldoen aan de **conditie** toegelaten.
- Wanneer je een **CHECK** constraint voor een **tabel** instelt, kan je combinatie van kolomwaardes beperken .
- Wordt niet in het ERD aangegeven


```
CREATE TABLE Auto (  
    nummerplaat CHAR(9) PRIMARY KEY CHECK(nummerplaat LIKE '_-_-_-_-'),  
    merk VARCHAR(255) NOT NULL,  
    eigenaarId INTEGER NOT NULL,  
    FOREIGN KEY (eigenaarId) REFERENCES Eigenaar(id)  
);
```

Table Voorbeeld

```
CREATE TABLE Auto (  
    nummerplaat CHAR(9) PRIMARY KEY,  
    merk VARCHAR(255) NOT NULL,  
    eigenaarId INTEGER NOT NULL,  
    FOREIGN KEY (eigenaarId) REFERENCES Eigenaar(id),  
    CONSTRAINT AUTO_nummerplaat_CC CHECK(nummerplaat LIKE '_-_-_-_-_-')  
);
```

Een constraint een naam
geven helpt later bij het
oplossen van errors.

Oefening MoreImmo

Maak een nieuwe tabel afspraken met de bijhorende PRIMARY KEY.

```
CREATE TABLE voorbeeld (  
    prim_col INT,  
    PRIMARY KEY(prim_col)  
);
```

Maak ook een DROP TABLE statement.

afspraken	
🔑	woning_id INT(11)
🔑	tijdstip TIMESTAMP
🔑	email VARCHAR(80)
Indexes	
PRIMARY	

Oefening moreImmo

Voeg een check constraint toe voor emailadres dat alleen odisee emailadressen zijn toegelaten.

Probeer enkele afpraken in te voeren om deze constraint te testen.

afspraken ▼	
🔑	woning_id INT(11)
🔑	tijdstip TIMESTAMP
◇	email VARCHAR(80)
Indexes ▼	
PRIMARY	

Table **NOT NULL**

- Standaard kan een kolom NULL waardes bevatten
- **NOT NULL** => NULL waardes zijn niet toegelaten
- In het ERD schema zijn alle velden welke NOT NULL mogen zijn volledig gekleurt, vb hire_date

```
◇ PHONE_NUMBER VARCHAR(20)  
◇ HIRE_DATE DATE
```

```
CREATE TABLE Eigenaar (  
    id INTEGER NOT NULL,  
    naam VARCHAR(255) NOT NULL,  
    email VARCHAR(255),  
    geboortedatum DATE  
);
```

Table UNIQUE

- Standaard zijn de waarden in een kolom niet uniek, d.w.z. een waarde mag meerdere keren voorkomen.
- **UNIQUE** => Alle waarden in een kolom moeten uniek zijn, uitgezonderd NULL

The screenshot displays the 'employees' table structure with the following columns and data types:

Column	Data Type
EMPLOYEE_ID	DECIMAL(6,0)
FIRST_NAME	VARCHAR(20)
LAST_NAME	VARCHAR(25)
EMAIL	VARCHAR(51)
PHONE_NUMBER	VARCHAR(20)
HIRE_DATE	DATE
JOB_ID	VARCHAR(10)
SALARY	DECIMAL(8,2)
COMMISSION_PCT	DECIMAL(2,2)
MANAGER_ID	DECIMAL(6,0)
DEPARTMENT_ID	DECIMAL(4,0)

Below the column list, the 'Indexes' section is expanded, showing the following index:

Index Name	Index Type
EMP_EMAIL_UK	PRIMARY

```
CREATE TABLE Eigenaar (  
    id INTEGER NOT NULL,  
    naam VARCHAR(255) NOT NULL,  
    email VARCHAR(255) UNIQUE,  
    geboortedatum DATE  
);
```


Foreign Key

- Een **FOREIGN KEY** is een key die gebruikt wordt om 2 tabellen te linken.
- Een **FOREIGN KEY** is een veld (of verzameling van velden) in een tabel die **REFEREREN** naar een **PRIMARY KEY** in andere tabel.
- De tabel met de foreign key wordt ook wel de child tabel genoemd. De tabel met de primary key wordt ook wel de parent tabel genoemd

```
◇ MANAGER_ID DECIMAL(6,0)  
◇ DEPARTMENT_ID DECIMAL(4,0)
```

Table
Voorbeeld

```
CREATE TABLE Auto (  
    nummerplaat CHAR(9) PRIMARY KEY,  
    merk VARCHAR(255) NOT NULL,  
    eigenaarId INTEGER NOT NULL,  
    FOREIGN KEY (eigenaarId) REFERENCES Eigenaar(id)  
);
```

Oefening moreImmo

Voor elke woning willen we maar 1 afspraak toestaan per emailadres. Maak een constraint zodat de combinatie van email+woning_id uniek is.

De woning_id bevat enkel waarden die voorkomen in de PK van woningen: woningen.id.

Leg de FK relatie tussen die 2 tabellen.

afspraken
woning_id INT(11)
tijdstip TIMESTAMP
email VARCHAR(80)
Indexes
PRIMARY

4. DDL extra's

- De **DEFAULT** wordt gebruikt om een **default value** voor een kolom in te stellen.
- De default value wordt gebruikt voor alle records **indien er geen andere waarde is gespecificeerd**.
- **Wordt niet in het ERD weergegeven**

```
CREATE TABLE Eigenaar (  
    id INTEGER PRIMARY KEY,  
    naam VARCHAR(255) NOT NULL DEFAULT 'John Doe',  
    email VARCHAR(255) UNIQUE,  
    geboortedatum DATE  
);
```

- Het **CREATE INDEX** statement wordt gebruikt om indexen toe te voegen aan een tabel.
- Met behulp van indexen kunnen we **data sneller** uit de database halen. De gebruiker ziet de indexen niet, deze zijn enkel aanwezig om **queries sneller** uit te voeren.
- Is geen constraint.
- Optimaliseert zoek queries, **vertraagt insert en update** queries
- Niet te kennen.

AUTO INCREMENT

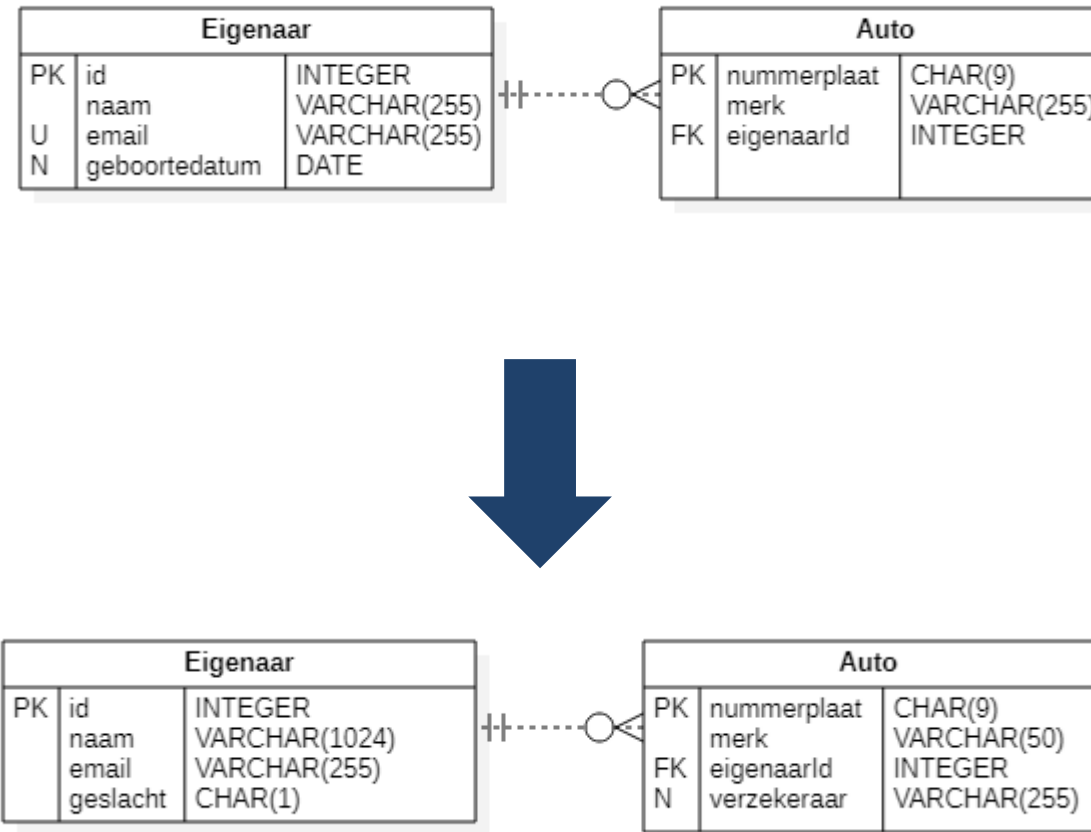
- **Auto-increment** genereert een **uniek number** wanneer er een nieuw item wordt toegevoegd aan de database
- **Vaak** is dit het **primary key** veld dat we automatisch willen laten genereren, telkens we een nieuw record is toegevoegd.
- Dit wordt niet in het ERD opgenomen.



5.

ALTER TABLE

DB wijzigingen



- Met het **ALTER TABLE** statement kunnen we kolomen toevoegen, verwijderen, of wijzigen in een bestaande tabel.
- Met het **ALTER TABLE** statement kunnen we ook constraints toevoegen en verwijderen in een bestaande tabel.

ALTER syntax

```
ALTER TABLE tabelnaam  
ADD kolomnaam datatype;
```

```
ALTER TABLE tabelnaam  
DROP COLUMN kolomnaam;
```

```
ALTER TABLE tabelnaam  
MODIFY COLUMN kolomnaam datatype;
```

ALTER Voorbeeld

```
ALTER TABLE eigenaar  
DROP COLUMN geboortedatum,  
ADD COLUMN geslacht CHAR(1) NOT NULL,  
MODIFY COLUMN naam VARCHAR(1024);
```

ALTER syntax

```
ALTER TABLE tabelnaam  
DROP CONSTRAINT constraintnaam;  
  
ALTER TABLE tabelnaam  
ADD CONSTRAINT constraint;
```

ALTER Voorbeeld

```
ALTER TABLE eigenaar  
DROP CONSTRAINT email;
```

Database Migraties

- Database migraties zijn **wijzigingen** die je moet uitvoeren op een database die reeds in gebruik is.
- Het **ALTER** statement in combinatie met geavanceerdere **INSERT INTO SELECT** en **UPDATE** statements is hier heel krachtig

OEFENING

- Gebruik **ALTER TABLE** om het constraint op **email+woning_id** te verwijderen.
- Voeg het daarna weer toe met **ALTER TABLE**.
- Afspraken: met **ALTER TABLE** voeg je een kolom medewerker toe.
- Indien bij het invoegen van een rij in afspraken er geen waarde wordt opgegeven voor medewerker dan moet er automatisch 'Suleiman de magnifieke' worden ingevuld.
Test dit.